

6. Algoritma Pengurutan (Sorting): Divide & Conquer

6.1. Tujuan Pembelajaran

Pertemuan ini bertujuan untuk:

- Memahami konsep Divide and Conquer.
- Mengimplementasikan Merge Sort dan Quick Sort menggunakan Python.
- Membandingkan proses dan hasil kedua algoritma.

6.2. Alat dan Bahan

Alat yang digunakan dalam praktikum:

- Komputer dan koneksi Internet.
- Code editor (Visual Studio Code, Pycharm atau sejenisnya).
- Windows Terminal / Powershell / sejenisnya.
- Python 3, Library Python standar

6.3. Materi

6.3.1. Divide and Conquer

Divide and Conquer adalah pendekatan penyelesaian masalah dengan cara membagi masalah yang besar menjadi beberapa masalah yang lebih kecil. Setiap bagian yang lebih kecil kemudian diselesaikan, dan hasilnya digunakan untuk memperoleh solusi dari masalah awal.

Secara umum, Divide and Conquer terdiri dari tiga tahap:

1. **Divide (Membagi):** Memecah masalah awal yang kompleks menjadi dua atau lebih sub-masalah yang lebih kecil dengan karakteristik yang serupa.
2. **Conquer (Menaklukkan):** Menyelesaikan tiap sub-masalah secara rekursif. Jika ukuran sub-masalah sudah sangat kecil (mencapai *base case*), masalah tersebut langsung diselesaikan secara sederhana tanpa pembagian lebih lanjut.
3. **Combine (Menggabungkan):** Menggabungkan seluruh solusi dari sub-masalah tadi menjadi satu solusi akhir yang utuh untuk masalah awal.

Pendekatan Divide and Conquer banyak digunakan dalam algoritma karena masalah yang besar dapat dipecah menjadi bagian-bagian yang lebih sederhana untuk diselesaikan. Dalam materi sorting, pendekatan Divide and Conquer digunakan pada **Merge Sort** dan **Quick Sort**.

Keunggulan Divide and Conquer:

- Cocok untuk masalah besar yang bisa dipisahkan secara rekursif.
- Implementasi relatif mudah.
- Meningkatkan efisiensi, terutama untuk algoritma sorting dan searching.

6.3.2. Merge Sort

Merge Sort menerapkan pembagian *array* secara konsisten tanpa bergantung pada nilai dari setiap elemen.

6.3.2.1. Proses Pembagian dan Rekursi

Array dibagi secara simetris tepat di titik tengah $mid = \lfloor n/2 \rfloor$ hingga menghasilkan sub-array tunggal. Proses pembagian ini membentuk struktur pohon rekursi dengan kedalaman maksimal $\log_2 n$.

Dengan kata lain Merge Sort bekerja dengan cara:

1. Membagi data menjadi dua bagian.
2. Membagi kembali setiap bagian sampai setiap bagian hanya memiliki satu elemen.
3. Menggabungkan kembali bagian-bagian tersebut dalam keadaan terurut.

6.3.2.2. Relasi Rekurens dan Kompleksitas Waktu

Persamaan rekurens untuk *Merge Sort* dinyatakan sebagai:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) \quad [1]$$

- $2T(n/2)$ mewakili waktu untuk menyelesaikan dua sub-masalah berukuran separuh.
- $\Theta(n)$ mewakili waktu yang dibutuhkan untuk proses penggabungan elemen (*merge*).

Berdasarkan Teorema Master, solusi persamaan tersebut menghasilkan kompleksitas waktu $O(n \log n)$ pada kondisi terbaik, rata-rata, maupun terburuk.

6.3.2.3. Analisis Ruang dan Stabilitas

- **Kompleksitas Ruang:** $O(n)$, karena membutuhkan alokasi memori tambahan berupa *array* sementara untuk melakukan pertukaran dan penggabungan elemen.
- **Sifat Stabilitas:** *Stable sort*. Algoritma ini menjamin posisi relatif dari elemen-elemen yang bernilai sama tidak akan berubah dari urutan awalnya.

6.3.3. Quick Sort

Quick Sort berfokus pada efisiensi partisi berbasis nilai elemen spesifik yang disebut *pivot*.

6.3.3.1. Mekanisme Partisi

Satu elemen dipilih sebagai *pivot*. Seluruh elemen lain ditata ulang sedemikian rupa sehingga:

- Elemen pada posisi sebelah kiri memiliki nilai lebih kecil dari atau sama dengan *pivot*.
- Elemen pada posisi sebelah kanan memiliki nilai lebih besar dari *pivot*.

Secara umum, Quick Sort bekerja dengan cara:

1. Memilih sebuah elemen sebagai *pivot*.
2. Membagi data menjadi bagian yang lebih kecil dan lebih besar dari *pivot*.
3. Mengurutkan bagian kiri dan kanan secara rekursif.
4. Menggabungkan hasilnya.

6.3.3.2. Kompleksitas Waktu Berdasarkan Pemilihan Pivot

- **Kondisi Terbaik dan Rata-rata:** Terjadi jika *pivot* selalu membagi *array* menjadi dua bagian yang seimbang.

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) \Rightarrow O(n \log n) \quad [2]$$

- **Kondisi Terburuk:** Terjadi jika *pivot* yang dipilih selalu merupakan nilai ekstrem (minimum atau maksimum), seperti pada *array* terurut dengan *pivot* di elemen pertama.

$$T(n) = T(n - 1) + \Theta(n) \Rightarrow O(n^2) \quad [3]$$

Terdapat perbedaan dengan Merge Sort yang memiliki kompleksitas waktu **$O(n \log n)$** . Perbedaan tersebut terjadi karena performa Quick Sort dipengaruhi oleh pemilihan *pivot*.

6.3.3.3. Analisis Ruang dan Stabilitas

- **Kompleksitas Ruang:** $O(\log n)$ pada kondisi rata-rata untuk mengelola penumpukan fungsi rekursif (*call stack*). *Quick Sort* beroperasi secara *in-place* tanpa *array* tambahan.
- **Sifat Stabilitas:** *Unstable sort*. Pertukaran elemen jarak jauh saat partisi dapat mengubah posisi relatif elemen yang bernilai sama.

6.3.4. Perbandingan Karakteristik Algoritma

Parameter Perbandingan	Merge Sort	Quick Sort
Strategi Pembagian	Berdasarkan posisi indeks tengah	Berdasarkan nilai elemen <i>pivot</i>
Kompleksitas Terburuk	$O(n \log n)$	$O(n^2)$
Penggunaan Memori Tambahan	$O(n)$	$O(\log n)$
Pendekatan Alokasi Memori	<i>Out-of-place</i>	<i>In-place</i>
Stabilitas Urutan	Stabil	Tidak Stabil

6.4. Kegiatan Praktikum

6.4.1. Kasus data tidak berurut

Pada praktikum ini, kita akan menyelesaikan sebuah permasalahan sederhana, yaitu mengurutkan sekumpulan data yang belum terurut. Data yang akan digunakan adalah:

data = [38, 12, 27, 43, 9, 31, 18, 25]

Data tersebut akan diurutkan dari nilai terkecil ke nilai terbesar (*ascending*) sehingga menghasilkan:

[9, 12, 18, 25, 27, 31, 38, 43]

Pada praktikum ini, permasalahan tersebut akan diselesaikan menggunakan: **Merge Sort** dan **Quick Sort**. Tujuan utama praktikum bukan hanya mendapatkan hasil akhir, tetapi memahami bagaimana data dibagi dan diproses oleh masing-masing algoritma.

1. Kita buat project baru dengan uv dengan nama `alstrukdat04`.
2. Kita buat file baru dengan nama `sorting_data.py`
3. Kita siapkan data, kita buat variabel data

```
data = [64, 34, 25, 12, 22, 11, 90]
print("Data Awal: ", data)
```

6.4.1.1. Implementasi Merge Sort

A. **Langkah 1** – Memahami Proses Divide

1. Membagi Data Menjadi Dua

Merge Sort tidak langsung membandingkan seluruh data.

Data awal: [38, 12, 27, 43, 9, 31, 18, 25]

Pertama-tama dibagi menjadi dua. Untuk melakukan pembagian tersebut, kita membutuhkan posisi tengah. Karena jumlah data ada 8, maka nilai mid = 4.

2. Selanjutnya Kita bagi data menjadi 2, dengan cara: (1) mengambil data dari awal sampai sebelum indeks 4; (2) mengambil data mulai indeks 4 sampai akhir.

[38, 12, 27, 43] [9, 31, 18, 25]

Code python menjadi

```
mid = len(data) // 2
left = data[:mid]
right = data[mid:]
print("Left :", left)
print("Right:", right)
```

B. **Langkah 2** - Data Dibagi Lagi

1. Pembagian data kita belum selesai. Data perlu dibagi-bagi kembali dengan cara yang sama seperti langkah 1. Sehingga data menjadi:

```
[38, 12, 27, 43, 9, 31, 18, 25]
      ↓
[38, 12, 27, 43]      [9, 31, 18, 25]
      ↓                      ↓
[38,12] [27,43]      [9,31] [18,25]
```

Tetapi kita tidak ingin menuliskan proses pembagian tersebut satu per satu secara manual di program. Oleh karena itu, Kita perlu program untuk melakukan pembagian berulang secara otomatis.

Dari kode tadi, kita ubah menjadi fungsi merge_sort().

```
def merge_sort(arr):
    mid = len(arr) // 2
    left = arr[:mid]
    right = arr[mid:]
```

2. Namun fungsi tersebut hanya membagi data sekali, sedangkan kita perlu membagi menjadi data yang sudah tidak bisa dipisah lagi. Karena proses pembagian dilakukan berulang dengan aturan yang sama, kita akan menggunakan **rekursif**.

Kita menggunakan rekursif karena setiap bagian data yang sudah dibagi masih harus diproses dengan aturan yang sama. Daripada menulis proses pembagian berulang-ulang secara manual, kita minta fungsi merge_sort() memanggil dirinya sendiri sampai mencapai kondisi berhenti

Rekursif terdapat 2 bagian:

a. **Recursive Case (Kasus Rekursif)**

Jika dilihat dari kode tadi, left dan right masih harus dipisah lagi. Oleh karena itu perlu memanggil dirinya sendiri. Artinya, setelah data dibagi menjadi left dan right, masing-masing bagian akan diproses dengan cara yang sama.

```
left = merge_sort(left)
right = merge_sort(right)
```

b. **Base Case (Kondisi Dasar)**

Sekarang muncul pertanyaan: **Kalau fungsi merge_sort() terus memanggil dirinya sendiri, kapan prosesnya berhenti?** Kita harus menentukan kapan rekursi berhenti. Pada merge kita perlu memisahkan data menjadi 1 saja sehingga tidak punya nilai lain untuk dibandingkan. **Mengapa harus sampai satu data?** Karena satu data dianggap sudah dalam kondisi terurut.

Jadi perlu kita tambahkan: Jika jumlah data masih lebih dari satu, proses pembagian dapat dilanjutkan. Tetapi jika jumlah data sudah **satu atau nol**, tidak perlu dibagi lagi.

```
if len(arr) <= 1:
    return arr
```

```
[38, 12, 27, 43, 9, 31, 18, 25]
      ↓
[38, 12, 27, 43]      [9, 31, 18, 25]
      ↓                ↓
[38, 12] [27, 43]      [9, 31] [18, 25]
      ↓                ↓                ↓
[38] [12] [27] [43]      [9] [31] [18] [25]
```

Dengan rekursi, kita tidak perlu menuliskan proses pembagian tersebut satu per satu.

3. Untuk dapat melihat proses pembagian, tambahkan print pada fungsi rekursif dan panggil fungsi tersebut

```
def merge_sort(arr):
    print("Divide:", arr)
    .....
    .....

    datadevide = merge_sort(data.copy())
    print("Data Akhir: ", datadevide)
```

Hasil:

```
Data Awal: [38, 12, 27, 43, 9, 31, 18, 25]
Divide: [38, 12, 27, 43, 9, 31, 18, 25]
Divide: [38, 12, 27, 43]
Divide: [38, 12]
Divide: [38]
Divide: [12]
Divide: [27, 43]
Divide: [27]
Divide: [43]
Divide: [9, 31, 18, 25]
Divide: [9, 31]
Divide: [9]
Divide: [31]
Divide: [18, 25]
Divide: [18]
Divide: [25]
Data Akhir: None
```

C. Langkah 3 – Masuk ke Proses Merge

Setelah proses pembagian berhenti, kita memiliki:

[38] [12] [27] [43] [9] [31] [18] [25]

Merge Sort mulai melakukan proses Combine, yaitu menggabungkan data kembali. Namun data tidak sekadar digabungkan. Data dibandingkan terlebih dahulu agar hasil penggabungan tetap terurut.

1. Tahap Pengurutan Array

a. Tahap 1: Penggabungan Array 1 Elemen menjadi Array 2 Elemen

- Merge [38] dan [12]
Bandingkan 38 dan 12 → 12 lebih kecil → Hasil: [12, 38]
- Merge [27] dan [43]
Bandingkan 27 dan 43 → 27 lebih kecil → Hasil: [27, 43]
- Merge [9] dan [31]
Bandingkan 9 dan 31 → 9 lebih kecil → Hasil: [9, 31]
- Merge [18] dan [25]
Bandingkan 18 dan 25 → 18 lebih kecil → Hasil: [18, 25]

Hasil Sementara Tahap 1: [12, 38], [27, 43], [9, 31], [18, 25]

b. Tahap 1: Penggabungan Array 2 Elemen menjadi Array 4 Elemen

- **Merge [12, 38] dan [27, 43]**
Bandingkan 12 dan 27 → Ambil 12 → Hasil: [12]
Bandingkan 38 dan 27 → Ambil 27 → Hasil: [12, 27]
Bandingkan 38 dan 43 → Ambil 38 → Hasil: [12, 27, 38]
Sisa elemen 43 langsung dimasukkan → Hasil: [12, 27, 38, 43]
- **Merge [9, 31] dan [18, 25]**
Bandingkan 9 dan 18 → Ambil 9 → Hasil: [9]
Bandingkan 31 dan 18 → Ambil 18 → Hasil: [9, 18]
Bandingkan 31 dan 25 → Ambil 25 → Hasil: [9, 18, 25]
Sisa elemen 31 langsung dimasukkan → Hasil: [9, 18, 25, 31]

Hasil Sementara Tahap 2: [12, 27, 38, 43] dan [9, 18, 25, 31]

c. Tahap 3: Penggabungan Akhir (Array 4 Elemen menjadi Array 8 Elemen)

- **Merge [12, 27, 38, 43] dan [9, 18, 25, 31]:**
Bandingkan 12 dan 9 → Ambil 9 → Hasil: [9]
Bandingkan 12 dan 18 → Ambil 12 → Hasil: [9, 12]
Bandingkan 27 dan 18 → Ambil 18 → Hasil: [9, 12, 18]
Bandingkan 27 dan 25 → Ambil 25 → Hasil: [9, 12, 18, 25]
Bandingkan 27 dan 31 → Ambil 27 → Hasil: [9, 12, 18, 25, 27]
Bandingkan 38 dan 31 → Ambil 31 → Hasil: [9, 12, 18, 25, 27, 31]
Sisa elemen di kelompok kiri (38 dan 43) dimasukkan sekaligus ke bagian akhir.

Hasil Akhir Terurut: [9, 12, 18, 25, 27, 31, 38, 43]

2. Untuk melakukan proses penggabungan, kita membuat fungsi merge().

```
def merge(left, right):
    result = []
    i = 0
    j = 0
```

Keterangan:

- left → bagian kiri yang sudah terurut.
- right → bagian kanan yang sudah terurut.
- result → tempat menyimpan hasil penggabungan.
- i → posisi data yang sedang diperiksa pada left.
- j → posisi data yang sedang diperiksa pada right.

3. Tambahkan proses perbandingan. Karena kita perlu mengurutkan data sebelum digabungkan

```
while i < len(left) and j < len(right):
    if left[i] < right[j]:
        result.append(left[i])
        i += 1
    else:
        result.append(right[j])
        j += 1
```

4. Kemudian kita gabungkan untuk tiap data.

```
result.extend(left[i:])
result.extend(right[j:])
return result
```

D. **Langkah 3** – Menghubungkan merge() dengan merge_sort()

Setelah left dan right selesai diproses oleh merge_sort(), keduanya sudah dalam keadaan terurut.

```
def merge_sort(arr):
    .....
    .....
    return merge(left, right)
```

E. Memahami Proses Lengkap Merge Sort

```
[38, 12, 27, 43, 9, 31, 18, 25]
      ↓
[38, 12, 27, 43]      [9, 31, 18, 25]
      ↓                ↓
[38, 12] [27, 43]      [9, 31] [18, 25]
      ↓                ↓                ↓
[38] [12] [27] [43]      [9] [31] [18] [25]
```

Setelah mencapai satu elemen, proses **Merge** dimulai:

```
[38] [12] [27] [43]      [9] [31] [18] [25]
      ↓                ↓                ↓
[12, 38] [27, 43]      [9, 31] [18, 25]
      ↓                ↓                ↓
[12, 27, 38, 43]      [9, 18, 25, 31]
      ↓
[9, 12, 18, 25, 27, 31, 38, 43]
```

F. Jalankan program yang telah dibuat.

```

Data Awal: [38, 12, 27, 43, 9, 31, 18, 25]
Divide: [38, 12, 27, 43, 9, 31, 18, 25]
Divide: [38, 12, 27, 43]
Divide: [38, 12]
Divide: [38]
Divide: [12]
Divide: [27, 43]
Divide: [27]
Divide: [43]
Divide: [9, 31, 18, 25]
Divide: [9, 31]
Divide: [9]
Divide: [31]
Divide: [18, 25]
Divide: [18]
Divide: [25]
Data Akhir: [9, 12, 18, 25, 27, 31, 38, 43]

```

6.4.1.2. Implementasi Quick Sort

Jika Merge Sort, data dibagi menjadi dua bagian. Pada **Quick Sort**, pendekatannya berbeda. Quick Sort memilih satu data sebagai **pivot**, kemudian data lainnya dibagi menjadi:

- data yang lebih kecil dari pivot
- pivot
- data yang lebih besar atau sama dengan pivot

Berbeda dengan Merge Sort yang melakukan proses **Divide terlebih dahulu sampai satu elemen**, Quick Sort melakukan pembagian berdasarkan **pivot**.

A. Langkah 1 – Menentukan Pivot

Kita menggunakan data: data = [38, 12, 27, 43, 9, 31, 18, 25]

Pertama kita kan memilih pivot terlebih dahulu. Pilihan pivot dapat memengaruhi jalannya partition dan performa Quick Sort, tetapi tidak mengubah tujuan akhirnya, yaitu menghasilkan data terurut. Terdapat beberapa pilihan umum untuk cara pemilihan pivot:

- Elemen pertama
- Elemen terakhir
- Elemen tengah
- Random – dipilih secara acak
- Median-of-three – median dari beberapa kandidat

Quick Sort dapat menggunakan berbagai cara memilih pivot. Pada praktikum ini kita menggunakan **elemen terakhir** agar prosesnya mudah dipahami.

```

data = [38, 12, 27, 43, 9, 31, 18, 25]
print("Data Awal: ", data)

pivot = data[-1]
print("Pivot ", pivot)

```

B. Langkah 2 – Partition

Setelah mendapatkan pivot, kita membandingkan setiap data dengan pivot. Partisi bertujuan untuk menemukan **posisi final (posisi asli yang tepat)** bagi satu elemen yang dipilih sebagai *pivot*, sekaligus memisahkan sisa elemen ke dalam dua kelompok independen: kelompok bernilai lebih kecil dan kelompok bernilai lebih besar.

Algoritma menyisir seluruh elemen dalam array dan membandingkan nilainya dengan pivot. Elemen yang lebih kecil dari pivot digeser/ditukar ke sisi kiri. Elemen yang lebih besar dari pivot dibiarkan atau digeser ke sisi kanan.

data = [38, 12, 27, 43, 9, 31, 18, 25]

pivot = 25

Sekarang kita lihat satu per satu, untuk data: 38, 12, 27, 43, 9, 31, 18. Tiap data akan kita bandingkan dengan pivot.

Bandingkan semua elemen dengan 25:

- Data 38 → 38 < 25? → Maka 38 masuk ke bagian kanan → right = [38]
- Data 12 → 12 < 25? → Maka 12 masuk ke bagian kiri → left = [12]
- Data 27 → 27 < 25? → Maka 27 masuk ke bagian kanan → right = [38,27]
- Data 43 → 43 < 25? → Maka 43 masuk ke bagian kanan → right = [38,27,43]
- Data 9 → 9 < 25? → Maka 9 masuk ke bagian kiri → left = [12,9]
- Data 31 → 31 < 25? → Maka 31 masuk ke bagian kanan → right = [38,27,43,31]
- Data 18 → 18 < 25? → Maka 18 masuk ke bagian kiri → left = [12,9,18]

Hasil partition:

Left	Pivot	Right
[12, 9, 18]	25	[38, 27, 43, 31]

Sekarang kita terjemahkan proses tersebut ke Python.

Kita siapkan dua list kosong.

left untuk menyimpan data yang lebih kecil dari pivot.

right untuk menyimpan data yang lebih besar atau sama dengan pivot.

Kemudian data awal akan dimasukkan satu-persatu, ke kanan atau kiri tergantung perbandingan dengan pivot. Kita melakukan perulangan pada semua data **kecuali pivot**.

```

left = []
right = []
for x in data[:-1]:
    if x < pivot:
        left.append(x)
    else:
        right.append(x)
print("Left :", left)
print("Pivot :", pivot)
print("Right :", right)

```

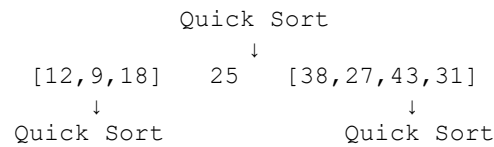
Data Awal: [38, 12, 27, 43, 9, 31, 18, 25]
Left : [12, 9, 18]
Pivot : 25
Right : [38, 27, 43, 31]

C. Langkah 3 – Ubah ke Rekursif?

Sampai tahap ini kita mendapatkan:

[12, 9, 18] 25 [38, 27, 43, 31]

Dapat terlihat bahwa left dan right belum terurut. Maka kita melakukan Quick Sort lagi terhadap kedua bagian tersebut. Oleh karena fungsi yang akan kita lakukan berulang, maka perlu dibuat rekursif.



1. Kita buat dahulu fungsi dari code yang sudah kita buat sebelumnya.

```
def quick_sort(arr):
    pivot = arr[-1]
    left = []
    right = []
    for x in arr[:-1]:
        if x < pivot:
            left.append(x)
        else:
            right.append(x)
```

2. **Base Case**

Seperti Merge Sort, Quick Sort juga membutuhkan kondisi untuk menghentikan recursion. Kita hentikan perulangan jika jmlah data adalah Satu. Jika data hanya memiliki 0 atau 1 elemen, data tersebut dianggap sudah terurut.

```
if len(arr) <= 1:
    return arr
```

3. **Rekursif Case**

Kita perhatikan alur dari proses quick sort adalah seperti ini:

- i. Tentukan pivot
- ii. Urutkan bagian kiri.
- iii. Letakkan pivot.
- iv. Urutkan bagian kanan.
- v. Gabungkan semuanya. → hasil adalah **left + pivot + right**

```
return quick_sort(left) + [pivot] + quick_sort(right)
```

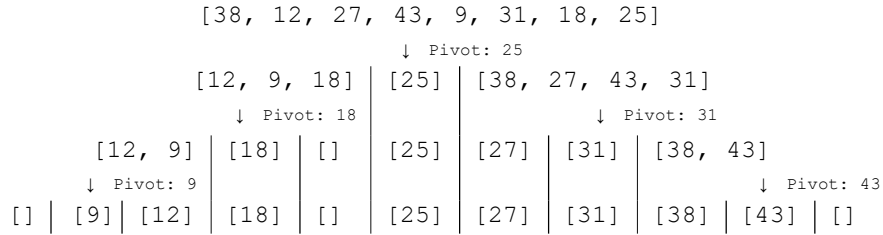
4. Agar proses Quick Sort dapat diamati, kita dapat menambahkan print().

```
def quick_sort(arr):
    print("Data:", arr)
    .....
    .....
    print("Left :", left)
    print("Pivot:", pivot)
    print("Right:", right)

    return quick_sort(left) + [pivot] + quick_sort(right)

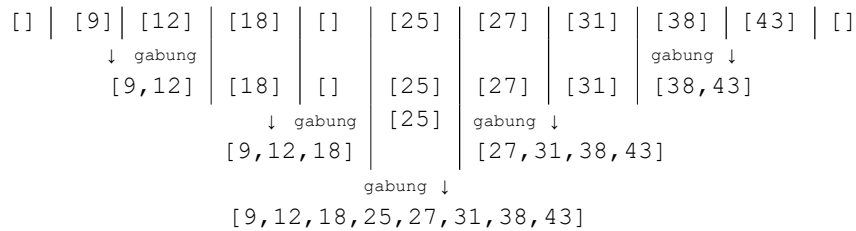
dataQuick = quick_sort(data.copy())
print("Data Akhir:", dataQuick)
```

D. Memahami Proses Lengkap Quick Sort



Setelah mencapai satu elemen, proses penggabungan berdasarkan **recursif case**.

quick_sort(left) + [pivot] + quick_sort(right)



E. Jalankan program yang telah dibuat.

```

Data Awal: [38, 12, 27, 43, 9, 31, 18, 25]
Data: [38, 12, 27, 43, 9, 31, 18, 25]
Left : [12, 9, 18]
Pivot: 25
Right: [38, 27, 43, 31]
Data: [12, 9, 18]
Left : [12, 9]
Pivot: 18
Right: []
Data: [12, 9]
Left : []
Pivot: 9
Right: [12]
Data: []
Data: [12]
Data: []
Data: [38, 27, 43, 31]
Left : [27]
Pivot: 31
Right: [38, 43]
Data: [27]
Data: [38, 43]
Left : [38]
Pivot: 43
Right: []
Data: [38]
Data: []
Data Akhir: [9, 12, 18, 25, 27, 31, 38, 43]

```

6.4.2. Studi Kasus

Sebuah perpustakaan memiliki data buku yang terdiri dari: **Judul**, **Tahun terbit**, **Penulis**, dan **Stok**. Data buku perlu diurutkan agar lebih mudah ditampilkan dan dicari.

Judul	Tahun	Penulis	Stok
Algoritma Pemrograman	2022	Andi Wijaya	5
Basis Data	2020	Andi Wijaya	3
Pemrograman Python	2023	Tono Pratama	7
Struktur Data	2021	Rina Putri	4
Jaringan Komputer	2019	Andi Wijaya	2
Rekayasa Perangkat Lunak	2022	Deni Kurniawan	6
Sistem Operasi	2020	Maya Sari	3
Kecerdasan Buatan	2024	Tono Pratama	5

Dalam penggunaannya, perpustakaan membutuhkan beberapa bentuk pengurutan data:

1. **Merge Sort** digunakan untuk menampilkan buku berdasarkan judul
2. **Quick Sort** digunakan untuk menampilkan buku berdasarkan tahun terbit

Catatan: Data bukunya tetap sama. Yang berbeda adalah **kriteria pengurutannya** dan algoritma yang digunakan.

1. Dari project dengan nama alstrukdat04, Kita buat file baru dengan nama `sorting_buku.py`
2. Kita siapkan data buku. Data tersebut menggunakan list yang berisi **dictionary**. Setiap dictionary merepresentasikan satu buku.

```
buku = [
    { "judul": "Algoritma Pemrograman", "tahun": 2022, "penulis": "Andi Wijaya", "stok": 5 },
    { "judul": "Basis Data", "tahun": 2020, "penulis": "Andi Wijaya", "stok": 3 },
    { "judul": "Pemrograman Python", "tahun": 2023, "penulis": "Tono Pratama", "stok": 7 },
    { "judul": "Struktur Data", "tahun": 2021, "penulis": "Rina Putri", "stok": 4 },
    { "judul": "Jaringan Komputer", "tahun": 2019, "penulis": "Andi Wijaya", "stok": 2 },
    { "judul": "Rekayasa Perangkat Lunak", "tahun": 2022, "penulis": "Deni Kurniawan", "stok": 6 },
    { "judul": "Sistem Operasi", "tahun": 2020, "penulis": "Maya Sari", "stok": 3 },
    { "judul": "Kecerdasan Buatan", "tahun": 2024, "penulis": "Tono Pratama", "stok": 5 }
]
```

3. Kita akan mengurutkan buku berdasarkan judul dengan Merge Sort
 Karena Merge Sort digunakan untuk mengurutkan berdasarkan judul, maka yang dibandingkan adalah: `buku["judul"]`. Python dapat membandingkan string berdasarkan urutan alfabet.
`if left[i]["judul"] < right[j]["judul"]:`

a. Langkah 1 - Membuat Fungsi Merge

Pada Merge Sort, kita membutuhkan fungsi `merge()` untuk menggabungkan dua bagian data yang **sudah terurut** menjadi satu bagian yang terurut.

Misalnya kita memiliki:

`left = [Basis Data, Struktur Data]`

right = [Jaringan Komputer, Sistem Operasi]

Prosesnya adalah:

- Bandingkan buku pertama dari left dan right.
- **Basis Data** dibandingkan dengan **Jaringan Komputer**.
- Karena B < J, maka Basis Data dimasukkan ke result.
- Selanjutnya bandingkan buku berikutnya.
- Proses dilanjutkan sampai salah satu bagian habis.
- Jika masih ada buku yang tersisa pada left atau right, buku tersebut ditambahkan ke result.

```
def merge(left, right):
    result = []
    i = 0
    j = 0
    while i < len(left) and j < len(right):
        if left[i]["judul"] < right[j]["judul"]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

b. Langkah 2 – Membuat Fungsi Merge Sort

Kita akan memecah list buku menjadi beberapa bagian yang lebih kecil. List dibagi menjadi dua bagian secara terus-menerus menggunakan rekursi sampai setiap bagian hanya memiliki satu buku.

Setelah itu, bagian-bagian kecil tersebut digabungkan kembali menggunakan fungsi merge() sambil diurutkan berdasarkan judul.

Data Awal	Pembagian Pertama	Pembagian 3	Pembagian -n Menyisakan 1 data
[Algoritma Pemrograman, Basis Data, Pemrograman Python, Struktur Data, Jaringan Komputer, Rekayasa Perangkat Lunak, Sistem Operasi, Kecerdasan Buatan]	[Algoritma Pemrograman, Basis Data, Pemrograman Python, Struktur Data] [Jaringan Komputer, Rekayasa Perangkat Lunak, Sistem Operasi, Kecerdasan Buatan]	[Algoritma Pemrograman, Basis Data] [Pemrograman Python, Struktur Data] [Jaringan Komputer, Rekayasa Perangkat Lunak] [Sistem Operasi, Kecerdasan Buatan]	[Algoritma Pemrograman] [Basis Data] [Pemrograman Python] [Struktur Data] [Jaringan Komputer] [Rekayasa Perangkat Lunak] [Sistem Operasi] [Kecerdasan Buatan]

```
def merge_sort(buku):
    if len(buku) <= 1:
        return buku

    mid = len(buku) // 2
    left = buku[:mid]
    right = buku[mid:]

    left = merge_sort(left)
    right = merge_sort(right)

    return merge(left, right)
```

Bagian penting dalam algoritma:

- Jika hanya ada satu buku, proses pembagian berhenti karena data tersebut sudah terurut.
`if len(buku) <= 1:`
- Menentukan posisi tengah untuk membagi data menjadi dua bagian.
`mid = len(buku) // 2`
- Membagi data menjadi bagian kiri dan kanan.
`left = buku[:mid]`
`right = buku[mid:]`
- Masing-masing bagian diurutkan kembali menggunakan Merge Sort.
`left = merge_sort(left)`
`right = merge_sort(right)`
- Dua bagian yang sudah terurut digabungkan menggunakan fungsi `merge()`.
`return merge(left, right)`

c. **Langkah 3 – Memanggil fungsi pengurutan**

Kita panggil fungsi `merge_sort()` untuk mengurutkan data buku berdasarkan judul.

```
hasil = merge_sort(buku)
print(
    f'{"Judul Buku":<30} '
    f'{"Penulis":<20} '
    f'{"Tahun Terbit"}')

for item in hasil:
    print(
        f'{item["judul"]:<30} '
        f'{item["penulis"]:<20} '
        f'{item["tahun"]}')
```

d. **Hasil pengurutan**

Dari hasil pengurutan tersebut, dapat dilihat bahwa buku telah tersusun berdasarkan judul secara alfabetis dari A sampai Z. Informasi tahun terbit, penulis, dan stok tetap mengikuti buku masing-masing.

Judul Buku	Penulis	Tahun Terbit
Algoritma Pemrograman	Andi Wijaya	2022
Basis Data	Andi Wijaya	2020
Jaringan Komputer	Andi Wijaya	2019
Kecerdasan Buatan	Tono Pratama	2024
Pemrograman Python	Tono Pratama	2023
Rekayasa Perangkat Lunak	Deni Kurniawan	2022
Sistem Operasi	Maya Sari	2020
Struktur Data	Rina Putri	2021

4. Kita akan mengurutkan buku berdasarkan tahun terbit menggunakan Quick Sort.
Kali ini yang dibandingkan bukan judul, tetapi: `item["tahun"]`
Contoh: $2020 < 2022 \rightarrow$ berarti buku tahun 2020 ditempatkan sebelum buku tahun 2022.

a. Langkah 1 – Memilih Pivot

Pada praktikum ini kita menggunakan elemen terakhir sebagai pivot. Jika dilihat dari data awal, elemen terakhir adalah: "Kecerdasan Buatan – 2024"
Maka pivot = 2024

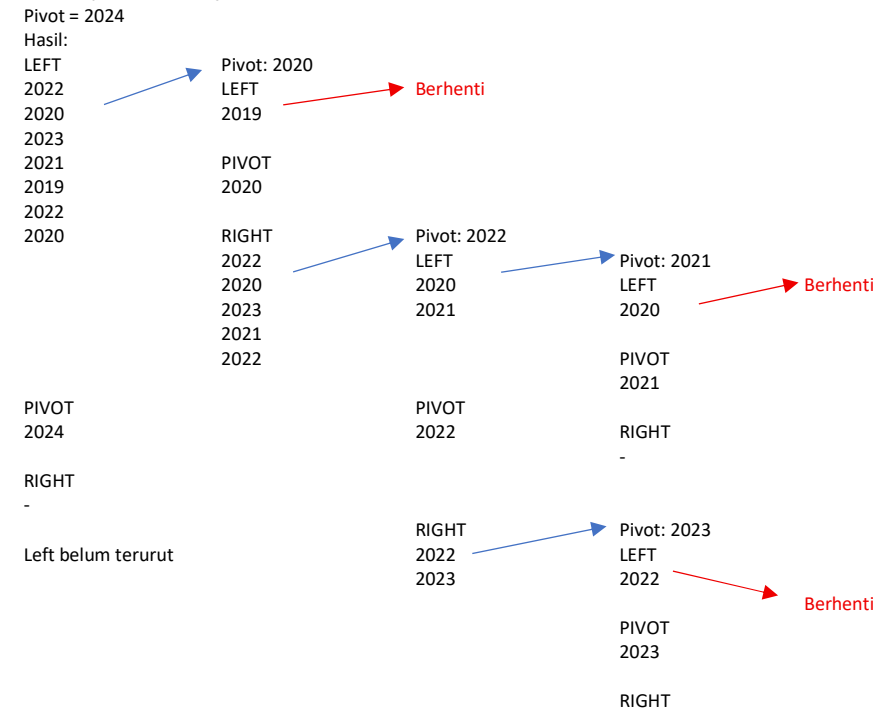
Pada praktikum ini, kita menggunakan **elemen terakhir dari list sebagai pivot**. Jika data masih memiliki lebih dari satu elemen, proses Quick Sort akan terus dilakukan secara rekursif dan pada setiap pemanggilan akan dipilih pivot baru.

b. Langkah 2 – Membagi Data Berdasarkan Pivot

Setiap buku selain pivot dibandingkan dengan pivot. Jika tahun terbit lebih kecil dari tahun pivot, buku dimasukkan ke left. Jika tidak, buku dimasukkan ke right.

```
for item in buku[:-1]:
    if item["tahun"] < pivot["tahun"]:
        left.append(item)
    else:
        right.append(item)
```

Proses partisi menjadi



Hasil partition pertama belum berarti data sudah terurut. Kita baru mengetahui bahwa semua data di left memiliki tahun lebih kecil dari 2024. Data di dalam left masih perlu diurutkan.

Quick Sort menggunakan fungsi rekursif. Agar rekursi berhenti, diperlukan kondisi berhenti atau **base case**.

```
if len(buku) <= 1:
    return buku
```

Artinya:

- Jika list kosong → tidak perlu diurutkan.
- Jika list hanya berisi satu buku → sudah dianggap terurut.
- Jika jumlah data lebih dari satu → proses partition dilakukan kembali.

c. Langkah 2 – Menggabungkan Hasil Partition

Pada Quick Sort, penggabungan ini sederhana karena data pada left sudah lebih kecil dari pivot dan data pada right sudah lebih besar atau sama dengan pivot.

Gabungkan LEFT + PIVOT + RIGHT. Proses dari bagian terkecil

Iterasi 3: - [2020, **2021**]

- [2022, **2023**]

Iterasi 2: - [2019, 2020, 2020, 2021, **2022**, 2022, 2023]

Iterasi 1: - [2019, 2020, 2020, 2021, 2022, 2023, **2024**]

Untuk efisiensi Langkah kita gunakan fungsi rekursif

```
def quick_sort(buku):
    if len(buku) <= 1:
        return buku

    pivot = buku[-1]
    left = []
    right = []
    for item in buku[:-1]:
        if item["tahun"] < pivot["tahun"]:
            left.append(item)
        else:
            right.append(item)

    return ( quick_sort(left) + [pivot] + quick_sort(right) )
```

d. Langkah 3 – Memanggil fungsi pengurutan

Kita panggil fungsi quick_sort() untuk mengurutkan data buku berdasarkan tahun terbit.

```
hasil = quick_sort(buku)
print(
    f'{"Judul Buku":<30} '
    f'{"Penulis":<20} '
    f'{"Tahun Terbit"}')
```

e. Hasil pengurutan

Dari hasil pengurutan tersebut, dapat dilihat bahwa buku telah tersusun berdasarkan **tahun terbit dari yang paling lama ke yang paling baru**. Informasi judul, penulis, dan stok tetap mengikuti data buku masing-masing.

Judul Buku	Penulis	Tahun Terbit
Jaringan Komputer	Andi Wijaya	2019
Sistem Operasi	Maya Sari	2020
Basis Data	Andi Wijaya	2020
Struktur Data	Rina Putri	2021
Rekayasa Perangkat Lunak	Deni Kurniawan	2022
Algoritma Pemrograman	Andi Wijaya	2022
Pemrograman Python	Tono Pratama	2023
Kecerdasan Buatan	Tono Pratama	2024

6.5. Latihan Mandiri

Setelah menyelesaikan program di atas, lakukan pengembangan berikut:

1. Ubahlah Merge Sort sehingga buku diurutkan berdasarkan penulis dari Z sampai A.
2. Ubahlah Quick Sort sehingga buku diurutkan berdasarkan tahun terbit dari terbaru ke terlama.
3. Buatlah program agar data bisa terurut berdasarkan penulis dan tahun secara ascending

Judul Buku	Penulis	Tahun Terbit
Andi Wijaya	Jaringan Komputer	2019
Andi Wijaya	Basis Data	2020
Andi Wijaya	Algoritma Pemrograman	2022
Deni Kurniawan	Rekayasa Perangkat Lunak	2022
Maya Sari	Sistem Operasi	2020
Rina Putri	Struktur Data	2021
Tono Pratama	Pemrograman Python	2023
Tono Pratama	Kecerdasan Buatan	2024

6.6. Reference

<https://medium.com/@rayhan.aditya.putra/implementasi-algoritma-sorting-abc5ce3b899f>

[https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2020-2021/Algoritma-Divide-and-Conquer-\(2021\)-Bagian1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2020-2021/Algoritma-Divide-and-Conquer-(2021)-Bagian1.pdf)

<https://www.dicoding.com/blog/teknik-algoritma-divide-and-conquer-greedy-dan-dynamic-programming/>

<https://www.geeksforgeeks.org/dsa/merge-sort/>